

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-a7b08b8bafea11eba.jsonl`
- SHA-256: `2be1faf82b30f019f71438d36e349e962d4b2cf6772dbf35318592fcd1ccd6f1`
- Source modified: `2026-06-22T09:50:29+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:48:51.241Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "Database(db_path) constructor + reel-count via len(segs) integration verified",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/infrastructure/database.py:24-27, nightly_reelcount.py:342, 351",
  "severity": "low",
  "detail": "Line 342 constructs Database(db_path) with a TemporaryDirectory path. Line 351 computes len(segs) where segs is the list returned by process_video(). No DB query needed ? the reel count IS len(segments_data) from the segmenter. The DB is populated but its segment count isn't directly read; the count comes from the segmenter's return list.",
  "fix": "No fix needed. Reel count logic is sound."
}
```

2. assistant (2026-06-22T09:48:53.228Z)

I'll analyze this finding by reading the relevant files to verify whether the reel-count integration is actually sound.

3. user (2026-06-22T09:48:54.194Z)

```
1      import os
2      import shutil
3      import sqlite3
4      import json
5      import logging
6      from contextlib import contextmanager
7      from typing import List, Dict, Any, Optional, Tuple
8
9      from backend import billing # pure cost math (no backend imports ? no cycle)
10
```

```

11     logger = logging.getLogger(__name__)
12
13     # The highest schema version this binary knows how to migrate TO (the top of the
migration ladder
14     # in `_init_db`). PACKAGING_PLAN.md P2d uses it for the upgrade-safety contract on a
downloadable,
15     # auto-updating app: refuse to open a DB from a NEWER app (downgrade guard) and back the
DB up before
16     # an upgrade migration runs.
`tests/test_database.py::test_schema_version_matches_ladder` pins it to
17     # the ladder so adding a `_migrate_to_vN` without bumping this fails CI.
18     SCHEMA_VERSION = 7
19
20
21     class SchemaTooNewError(RuntimeError):
22         """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
23
24     class Database:
25         def __init__(self, db_path: str):
26             self.db_path = db_path
27             self._init_db()
28
29         @contextmanager
30         def _connect(self):
31             """Transaction scope that also CLOSES the connection.
32
33             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
34             never closes, so every call leaked its connection to GC (lingering file
35             handles on the WAL sidecars under Windows). Internal methods use this.
36             """
37             conn = self._get_connection()
38             try:
39                 with conn:
40                     yield conn
41             finally:
42                 conn.close()
43
44         def _get_connection(self):
45             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
46             # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
47             conn = sqlite3.connect(self.db_path, timeout=30.0)
48             conn.row_factory = sqlite3.Row
49             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
50             conn.execute("PRAGMA foreign_keys = ON")

```

4. user (2026-06-22T09:48:54.864Z)

```

1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3     Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5     vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6     trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7     needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8     owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9

```

```

10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see Sguard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02 # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).

```

```

59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
76         arithmetic).
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
78         ``--vm-uptime-seconds``;
79         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
80         for boot/install)."""
81         rate_per_sec = float(machine_usd_per_hr) / 3600.0
82         usage = {billing.WALL_SECONDS: float(billed_seconds)}
83         rates = {billing.WALL_SECONDS: rate_per_sec}
84         usd, breakdown = billing.compute_cost_usd(usage, rates)
85         return {
86             "billed_seconds": round(float(billed_seconds), 1),
87             "gpu_seconds": round(float(gpu_seconds), 1),
88             "machine_usd_per_hr": float(machine_usd_per_hr),
89             "usd": usd,
90             "inr": billing.to_inr(usd, fx_inr_per_usd),
91             "breakdown": breakdown,
92         }
93
94     # ----- #
95     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
96     # ----- #
97     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
98                          cost: Dict[str, Any]) -> Dict[str, Any]:
99         """Compact, comparable snapshot from per-video run results.
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
101         wall_seconds}``.
102         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
103         never hides it)."""
104         per_video: Dict[str, Any] = {}
105         for r in run_results:
106             per_video[r["name"]] = {
107                 "reel_count": r.get("reel_count"),
108                 "ok": bool(r.get("ok", False)),
109                 "error": r.get("error"),
110                 "quality": r.get("quality"),
111             }
112         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
113                "n": len(per_video)}
114
115     @dataclass
116     class Finding:
117         video: str
118         metric: str
119         # "reel_count" | "error" | a quality key | "segmenter" |
120         "corpus"

```

```

118         kind: str                                # "regression" | "improvement" | "stale"
119         baseline: Optional[float] = None
120         current: Optional[float] = None
121         detail: str = ""
122
123     def message(self) -> str:
124         if self.kind == "stale":
125             return f"[STALE] {self.metric}: {self.detail}"
126         if self.metric == "error":
127             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
128         b = "?" if self.baseline is None else f"{self.baseline:g}"
129         c = "?" if self.current is None else f"{self.current:g}"
130         tag = "REGRESSION" if self.kind == "regression" else "improvement"
131         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
132
133
134     @dataclass
135     class NightlyResult:
136         findings: List[Finding] = field(default_factory=list)
137         added: List[str] = field(default_factory=list)
138         removed: List[str] = field(default_factory=list)
139
140         @property
141         def regressions(self) -> List[Finding]:
142             return [f for f in self.findings if f.kind == "regression"]
143
144         @property
145         def improvements(self) -> List[Finding]:
146             return [f for f in self.findings if f.kind == "improvement"]
147
148         @property
149         def stale(self) -> List[Finding]:
150             return [f for f in self.findings if f.kind == "stale"]
151
152         def overall_status(self) -> str:
153             if self.regressions:
154                 return "REGRESSION"
155             if self.stale:
156                 return "STALE"
157             return "PASS"
158
159         def exit_code(self) -> int:
160             if self.regressions:
161                 return 1
162             if self.stale:
163                 return 4
164             return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))

```

```

179         return res
180
181     b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182     res.added = sorted(set(c_pv) - set(b_pv))
183     res.removed = sorted(set(b_pv) - set(c_pv))
184     if res.added or res.removed:
185         res.findings.append(Finding("", "corpus", "stale",
186                                   detail=f"added={res.added} removed={res.removed}"))
187
188     for v in sorted(set(b_pv) & set(c_pv)):
189         b, c = b_pv[v], c_pv[v]
190         # 1) pipeline must still succeed
191         if not c.get("ok", False) or c.get("reel_count") is None:
192             res.findings.append(Finding(v, "error", "regression",
193                                       detail=str(c.get("error") or "no reel_count
194 produced"))))
195             continue
196         # 2) reel count ? EXACT
197         bc, cc = b.get("reel_count"), c.get("reel_count")
198         if bc is not None and cc is not None and cc != bc:
199             res.findings.append(Finding(v, "reel_count", "regression", float(bc),
200 float(cc)))
201         # 3) quality metrics ? tolerance (only if both sides have them)
202         bq, cq = b.get("quality") or {}, c.get("quality") or {}
203         for m in QUALITY_HIGHER:
204             bv, cv = bq.get(m), cq.get(m)
205             if bv is None or cv is None:
206                 continue
207             if cv < bv - tols.quality_tol:
208                 res.findings.append(Finding(v, m, "regression", bv, cv))
209             elif cv > bv + tols.quality_tol:
210                 res.findings.append(Finding(v, m, "improvement", bv, cv))
211
212     return res
213
214 # ----- #
215 # Report formatting (pure).
216 # ----- #
217 def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
218     if not qd or qd.get(key) is None:
219         return "?"
220     return f"{qd[key]:.3f}"
221
222 def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
223                  result: NightlyResult, date: str, head: str) -> str:
224     status = result.overall_status()
225     badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
226             "STALE": "? STALE (re-freeze the baseline)"}[status]
227     cost = current.get("cost", {})
228     L: List[str] = [
229         f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
230         "",
231         f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
232 {current.get('segmenter')}",
233         "",
234         f"- HEAD {head} · baseline {baseline.get('git_commit', '?')} (frozen
235 {baseline.get('generated_at', '?')})",
236         f"- **Run cost: ${cost.get('usd', 0):.4f} ({cost.get('inr', 0):.2f})** · "
237         f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
238 0)}/hr",
239         "",
240     ]
241     if result.regressions:
242         L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]

```

```

+ [""]
239         if result.stale:
240             L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241             L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243         # Per-video table.
244         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245         L += ["## Per-video", "",
246             "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247             "|---|---|---|---|---|"]
248         for v in sorted(set(b_pv) | set(c_pv)):
249             b, c = b_pv.get(v, {}), c_pv.get(v, {})
250             bc = b.get("reel_count")
251             cc = c.get("reel_count")
252             rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253             bq, cq = b.get("quality"), c.get("quality")
254             tf = f"_q(bq, 'tol_f1')?_q(cq, 'tol_f1')]"
255             f5 = f"_q(bq, 'f1@0.5')?_q(cq, 'f1@0.5')]"
256             vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257             L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258             L.append("")
259
260         if result.improvements:
261             L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
+ [f"- {f.message()}" for f in result.improvements] + [""]
262         L += ["---",
263             "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
264             "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
265         return "\n".join(L)
266
267
268
269         # ----- #
270         # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271         # ----- #
272         def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273             """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274             (`start,end` columns / pairs). Returns None when no labels are configured."""
275             if not labels_ref:
276                 return None
277             from backend.storage import fetch
278             local = fetch(labels_ref)
279             if local.endswith(".json"):
280                 with open(local, encoding="utf-8") as fh:
281                     data = json.load(fh)
282                     return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283             # CSV: header start,end (or first two numeric columns)
284             import csv
285             out: List[Tuple[float, float]] = []
286             with open(local, encoding="utf-8") as fh:
287                 for row in csv.reader(fh):
288                     try:
289                         out.append((float(row[0]), float(row[1])))
290                     except (ValueError, IndexError):
291                         continue
292             return out
293
294
295         def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:

```

```

296         """"(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
298         optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
316         reproduce, a real change will."""
317         import random
318         import time
319
320         from backend.eval import rally_seg_eval
321         from backend.infrastructure.database import Database
322         from backend.pipeline.segmenters import segmenter_registry
323         from backend.results import Failure
324         from backend.storage import fetch
325         from backend.utils.hashing import compute_video_id
326
327         name = video["name"]
328         try:
329             local = fetch(video["uri"])
330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                    "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)
346             wall = time.monotonic() - t0
347             if isinstance(result, Failure):
348                 return None, None, wall, getattr(result, "message", "segmenter failed")
349             segs = result.value if hasattr(result, "value") else result
350             intervals = _segments_to_intervals(segs)

```



```

351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
367 value
368
369         quality = None
370         if gts and intervals is not None:
371             row = rally_seg_eval.score_intervals(intervals, gts)
372             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
373 "tol_precision", "tol_recall")}
374             if k in row}
375
376         return {"name": name, "reel_count": count, "ok": True, "error": None,
377                "quality": quality, "wall_seconds": round(total_wall, 2)}
378
379     def load_manifest(path: str) -> Dict[str, Any]:
380         with open(path, encoding="utf-8") as fh:
381             return json.load(fh)
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
386 total_wall_s)."""
387         from backend.config import load_config
388
389         config = load_config()
390         # apply manifest config overrides onto the typed config (dotted keys)
391         for dotted, val in (manifest.get("config_overrides") or {}).items():
392             _set_dotted(config, dotted, val)
393         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
394 "default_segmenter", "motion")
395         b_pv = (baseline or {}).get("per_video", {})
396         results: List[Dict[str, Any]] = []
397         total_wall = 0.0
398         for v in manifest.get("videos", []):
399             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
400             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
401             total_wall += float(r.get("wall_seconds") or 0.0)
402             results.append(r)

```

5. user (2026-06-22T09:48:55.781Z)

```

1     """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
2 Manager.
3     Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
4 Deliberately
5     dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
6 password can
7     be resolved it logs + returns ``False`` (the report is still written/printed ? a broken

```

```

mailer must
6     never mask a regression).
7
8     Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9     ? Recipient: ``--email-to`` arg ? ``NIGHTLY_EMAIL_TO`` ? ``avi.dullu@gmail.com``
10    ? SMTP host: ``NIGHTLY_SMTP_HOST`` (default ``smtp.gmail.com``) . port
    ``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11    ? SMTP user: ``NIGHTLY_SMTP_USER`` (default = recipient)
12    ? SMTP pass: ``NIGHTLY_SMTP_PASS`` ? Secret Manager ``NIGHTLY_SMTP_SECRET``
13                (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14                lib if installed, else the ``gcloud`` CLI). A Gmail app password is
the simplest.
15
16    Set the secret once (owner): echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17    and on the VM: export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19    ""
20    from __future__ import annotations
21
22    import logging
23    import os
24    from email.message import EmailMessage
25    from typing import Optional
26
27    log = logging.getLogger("nightly_email")
28
29    DEFAULT_TO = "avi.dullu@gmail.com"
30    DEFAULT_HOST = "smtp.gmail.com"
31    DEFAULT_PORT = 587
32
33
34    def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35        """"A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable.""
36        msg = EmailMessage()
37        msg["Subject"] = subject
38        msg["From"] = from_addr
39        msg["To"] = to_addr
40        msg.set_content(body_md)
41        return msg
42
43
44    def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45        """"Resolve the SMTP password from env or GCP Secret Manager. Returns None if
unresolvable.""
46        env = env if env is not None else os.environ
47        direct = env.get("NIGHTLY_SMTP_PASS")
48        if direct:
49            return direct
50        secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51        if not secret_ref:
52            return None
53        # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54        try:
55            from google.cloud import secretmanager # type: ignore
56            client = secretmanager.SecretManagerServiceClient()
57            return
58            client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
59        except Exception: # noqa: BLE001 ? lib absent or auth issue ? try the CLI
            pass

```

```

60         try:
61             import subprocess
62             name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63             out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                   capture_output=True, text=True, timeout=30)
65             return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66         except Exception as e: # noqa: BLE001
67             log.warning("secret-manager resolution failed: %s", e)
68             return None
69
70
71     def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                   env: Optional[dict] = None) -> bool:
73         """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74         env = env if env is not None else os.environ
75         to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76         host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77         port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78         user = env.get("NIGHTLY_SMTP_USER") or to_addr
79         password = resolve_secret(env)
80         if not password:
81             log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82             return False
83
84         import smtplib
85         msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86         with smtplib.SMTP(host, port, timeout=60) as s:
87             s.starttls()
88             s.login(user, password)
89             s.send_message(msg)
90         log.info("emailed report to %s", to_addr)
91         return True
92

```

6. assistant (2026-06-22T09:48:57.907Z)

Now let me examine the specific lines mentioned in the finding and verify the actual flow:

7. user (2026-06-22T09:48:59.844Z)

```

330         except Exception as e: # noqa: BLE001 ? surface, never hide
331             return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
332                     "quality": None, "wall_seconds": 0.0}
333
334         gts = _load_gts(video.get("labels_uri"))
335         seg_cls = segmenter_registry.get(segmenter)
336         if not seg_cls:
337             return {"name": name, "reel_count": None, "ok": False,
338                     "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}
339
340         def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
float]]], float, Optional[str]]:
341             random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
342             db = Database(db_path)
343             video_id = compute_video_id(local)
344             t0 = time.monotonic()
345             result = seg_cls(db, config).process_video(video_id, local)

```

```

346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")
349         segs = result.value if hasattr(result, "value") else result
350         intervals = _segments_to_intervals(segs)
351         return len(segs), intervals, wall, None
352
353     import tempfile
354     total_wall = 0.0
355     with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
356         count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
357         total_wall += wall
358         if err is not None:
359             return {"name": name, "reel_count": None, "ok": False, "error": err,
360                    "quality": None, "wall_seconds": round(total_wall, 2)}
361         # re-run-once guard for the transient DB-drop flake
362         if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
363             count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
364             total_wall += wall2
365             if err2 is None and count2 is not None:
366                 count, intervals = count2, intervals2 # the reproduced (or corrected)
value
367
368         quality = None
369         if gts and intervals is not None:
370             row = rally_seg_eval.score_intervals(intervals, gts)
371             quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")}
372                 if k in row}
373
374         return {"name": name, "reel_count": count, "ok": True, "error": None,
375                "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
386         from backend.config import load_config
387
388         config = load_config()
389         # apply manifest config overrides onto the typed config (dotted keys)
390         for dotted, val in (manifest.get("config_overrides") or {}).items():
391             _set_dotted(config, dotted, val)
392         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
393         b_pv = (baseline or {}).get("per_video", {})
394         results: List[Dict[str, Any]] = []
395         total_wall = 0.0
396         for v in manifest.get("videos", []):
397             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
398             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
399             total_wall += float(r.get("wall_seconds") or 0.0)
400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""

```

```

406         parts = dotted.split(".")
407         obj = config
408         for p in parts[:-1]:
409             obj = getattr(obj, p, None)

```

8. user (2026-06-22T09:49:00.766Z)

```

400             results.append(r)
401         return results, total_wall
402
403
404     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
405         """Best-effort dotted-path set onto a pydantic config (e.g.
406         'indexing.motion_threshold')."""
407         parts = dotted.split(".")
408         obj = config
409         for p in parts[:-1]:
410             obj = getattr(obj, p, None)
411             if obj is None:
412                 return
413         try:
414             setattr(obj, parts[-1], value)
415         except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
416             crash the run
417             pass
418
419     def _git_head() -> str:
420         import subprocess
421         try:
422             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
423                                 capture_output=True, text=True, timeout=10)
424             return out.stdout.strip() or "?"
425         except Exception: # noqa: BLE001
426             return "?"
427
428     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
429         if not os.path.exists(path):

```

9. assistant (2026-06-22T09:49:03.059Z)

Now let me check what process_video returns to understand if the reel count logic is truly correct:

10. user (2026-06-22T09:49:04.100Z)

```

backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\_keyhint.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py

```

11. user (2026-06-22T09:49:06.577Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==

```

```

Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60                 res[name] = desc
61             return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

Now let me check what the actual process_video implementations return to see if there's a DB query issue:

13. user (2026-06-22T09:49:10.230Z)

```
1      """WASB-based hybrid play segmenter.
2
3      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5      badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7      For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8      candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9      min_window_duration) are what get tuned against golden rally labels.
10
11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
34             badminton "
35             "weights) for precise candidate rallies + AI provider for semantic
36             boundaries.")
37
38         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
39         Any]]:
40             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
41             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
42
43         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
44         Dict[str, Any]]:
45             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
46             stub
47             # seam (imported only when detector_impl='native' is actually selected).
48             from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
49             NativeWasbConfig
50             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
51             return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
52             "detector_impl": "native", "device": cfg.device}
```

14. user (2026-06-22T09:49:12.737Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.
```

```

2
3 tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4 standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5 now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10 A concrete subclass supplies only its identity and runner wiring:
11 - ``family`` ? config section under ``indexing.*``, output subdir, and
12   the ``<family>-hybrid-<model>`` detector tag in metadata.
13 - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14 - ``name`` / ``description`` properties (the registry contract).
15 - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16   detection_params extras such as weights_path).
17
18 This base is deliberately NOT registered; only concrete subclasses register.
19 """
20 import os
21 import time
22 import logging
23 import concurrent.futures
24 from typing import Any, Dict, List, Optional, Tuple
25
26 import cv2
27
28 from backend.pipeline.segmenters.base import BasePlaySegmenter
29 from backend.utils.video import get_video_duration, extract_video_chunk
30 from backend.utils.paths import output_base
31 from backend.sports import sport_registry
32 from backend.providers import provider_registry
33 from backend.pipeline.detectors.base import DetectorRunner
34
35 logger = logging.getLogger(__name__)
36
37
38 def _fmt_ts(seconds: float) -> str:
39     seconds = max(0, int(seconds))
40     h, rem = divmod(seconds, 3600)
41     m, s = divmod(rem, 60)
42     return f"{h:02d}:{m:02d}:{s:02d}"
43
44
45 class TrajectoryHybridSegmenter(BasePlaySegmenter):
46     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
47 boundaries."""
48     #: config section under `indexing` (velocity_threshold lives there), the
49     #: output subdir for trajectories, and the metadata detector-tag prefix.
50     family: str = ""
51     #: human-readable label used in log lines.
52     display_label: str = ""
53
54     def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56         """Return (runner, detector-specific detection_params extras) for the default
57 (WSL) path."""
58         raise NotImplementedError
59
60     def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
61 Dict[str, Any]]:
62         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
63 with a
64 native runner override this; the base refuses with a clear message (M3.5: WASB
65 only)."""

```



```

61         raise NotImplementedError(
62             f"detector_impl='native' is not supported for the "
63             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
64             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
65
66     def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
67         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
68
69         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
70         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
71         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
72         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
73         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
74         (the WSL runner) so the default production path is unchanged
75         (``docs/DECOUPLED_COMPUTE/``).
76         """
77         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
78         if impl == "stub":
79             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
80             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
self.display_label or "trajectory")
84             return self._build_native_runner(idx_cfg)
85         return self._build_runner(idx_cfg)
86
87     @staticmethod
88     def _probe_fps_width(video_path: str) -> tuple:
89         """Return (fps, frame_width) for a video, with sensible fallbacks."""
90         cap = cv2.VideoCapture(video_path)
91         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
92         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
93         cap.release()
94         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
95
96     @staticmethod
97     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
98         """Provider-reported exchange count, else a duration-based estimate.
99
100         One canonical implementation ? the twins had drifted (wasb relied on
``int(None)`` raising into the fallback; same result, by accident).
101         """
102         shot_count = segment.get("shuttle_exchange_count")
103         if shot_count is None:
104             return max(3, int(duration / 0.8))
105         try:
106             return int(shot_count)
107         except (ValueError, TypeError):
108             return max(3, int(duration / 0.8))
109
110     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
111     -----
112     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,

```

```

115             frame_width: float, video_path: str,
116             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
117         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
118         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
119         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
120         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
121         return {
122             "peak_velocity": cw["peak_velocity"],
123             "mean_velocity": cw["mean_velocity"],
124             "active_frame_count": cw["active_frame_count"],
125             "track_id_count": cw["track_id_count"],
126         }
127
128     def _select_for_handoff(self, windows: List[Dict[str, Any]],
129                           window_stats: List[Dict[str, Any]],
130                           idx_cfg: Dict[str, Any]) -> List[int]:
131         """Indices of the candidate windows that proceed to the AI handoff (and thus may
132         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
133         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
134         Persisted candidates are NOT affected ? they remain the full superset for
offline
135         re-scoring."""
136         return list(range(len(windows)))
137
138     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
139                      player_pool: Optional[List[str]] = None,
140                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
141         # override-ignore: the ABC declares the Result contract (Success|Failure)
142         # but every live segmenter still returns a bare list ? the documented
143         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
144         label = self.display_label
145         # --- Sport profile + AI provider wiring (identical across segmenters) ---
146         sport_name = self.config.get("sport", "badminton")
147         try:
148             sport_profile = sport_registry.get(sport_name)
149         except KeyError as e:
150             logger.error(str(e))
151         return []
152
153         idx_cfg = self.config.get("indexing", {})
154         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
155         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
156         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
157         # measure it against the Gemini path, or to avoid the cloud entirely). With
158         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
159         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
160         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
161         if skip_ai:
162             model_name = "local-noai"
163             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
164                         "be emitted as rallies; no %s handoff, no API key needed.",
165                         label, provider_name)
166         else:
167             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
168             api_key_env_var = f"{provider_name.upper()}_API_KEY"

```

```

169         api_key = os.environ.get(api_key_env_var)
170         if not api_key:
171             from backend.pipeline.segmenters._keyhint import api_key_hint
172             logger.error(
173                 f"{api_key_env_var} environment variable is not set! "
174                 f"To run the {provider_name} handoff, set your API key. "
175                 + api_key_hint(api_key_env_var)
176             )
177             return []
178         try:
179             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
180         except KeyError as e:
181             logger.error(str(e))
182             return []
183
184         video_duration = get_video_duration(video_path)
185         if video_duration <= 0:
186             logger.error("Invalid video duration.")
187             return []
188         fps, frame_width = self._probe_fps_width(video_path)
189
190         # --- Runner config + windowing thresholds ---
191         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
192         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
193         # it delegates to the subclass _build_runner (the real WSL/native runner).
194         try:
195             runner, runner_params = self._resolve_runner(idx_cfg)
196         except NotImplementedError as e:
197             # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
198             # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
199             logger.error("%s: %s", label, e)
200             return []
201
202         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
203         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
204         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
205         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
206         max_window_duration = idx_cfg.get("max_window_duration", 0.0)
207         window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
208         window_hard_cap = idx_cfg.get("window_hard_cap", False)
209         window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
210         inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
211         inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
212         window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
213         window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
214         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
215         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
216         log_candidates = idx_cfg.get("log_yolo_candidates", True)
217         store_candidates = idx_cfg.get("store_yolo_candidates", True)
218
219         detection_params = {
220             "detector": self.name,
221             "velocity_thresh": velocity_thresh,
222             "merge_gap": merge_gap,
223             "min_action_window_duration": min_window_duration,
224             **runner_params,
225         }
226
227         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
228         ok, msg = runner.healthcheck()
229         if not ok:

```

```

230         logger.error("%s detector environment not ready: %s", label, msg)
231         return []
232     logger.info("%s detector env OK: %s", label, msg)
233
234     # --- Phase 2: trajectory -> candidate rally windows ---
235     # Trajectory detectors process the whole video in one pass (they carry
236     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
237     t_start = time.time()
238     out_dir = os.path.join(output_base(self.config), self.family)
239     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
240     if not csv_path:
241         logger.error("%s produced no trajectory; aborting.", label)
242         return []
243
244     points = runner.parse_trajectory_csv(csv_path)
245     logger.info("Parsed %d trajectory points (%d visible).",
246               len(points), sum(1 for p in points if p.visible))
247
248     all_candidate_windows = runner.trajectory_to_action_windows(
249         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
250         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
251         min_window_duration=min_window_duration, chunk_index=0,
252         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
253         window_hard_cap=window_hard_cap,
254         window_inactivity_end=window_inactivity_end,
255         inactivity_rally_thresh=inactivity_rally_thresh,
256         inactivity_min_density=inactivity_min_density,
257         window_blob_fallback=window_blob_fallback,
258         window_blob_factor=window_blob_factor,
259     )
260     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
261               label, time.time() - t_start, len(all_candidate_windows))
262
263     if log_candidates:
264         for cw in all_candidate_windows:
265             logger.info(f"[{label} rally]
266             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
267             f"({cw['end'] - cw['start']:.1f}s) |
268             frames={cw['active_frame_count']} "
269             f"peak_v={cw['peak_velocity']:.3f}
270             mean_v={cw['mean_velocity']:.3f}")
271
272     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
273     fusion
274     # segmenter adds net_crossings + person-cue features. Computed once, reused for
275     both
276     # persistence and the handoff gate below.
277     window_stats = [
278         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
279         idx_cfg)
280         for cw in all_candidate_windows
281     ]
282
283     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
284     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
285     if store_candidates:
286         for i, cw in enumerate(all_candidate_windows):
287             candidate_ids[i] = self.db.add_candidate_segment(
288                 video_id, detector=self.name,
289                 start_time=cw["start"], end_time=cw["end"],
290                 chunk_index=cw["chunk_index"], confidence=min(1.0,
291                 cw["peak_velocity"]),
292                 stats=window_stats[i], detection_params=detection_params,
293             )

```

```

288         if not all_candidate_windows:
289             return []
290
291         # --- Phase 3: AI provider handoff over padded candidate clips ---
292         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
293         # candidates above stay the full superset ? only the served play_segments are
gated.
294         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
295
296         ai_segments: List[dict] = []
297         if skip_ai:
298             # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
299             # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
300             # falls back to the duration-based estimate (no AI exchange count).

```

15. user (2026-06-22T09:49:16.317Z)

```

300         # falls back to the duration-based estimate (no AI exchange count).
301         ai_segments = [
302             {
303                 "start_time": all_candidate_windows[wi]["start"],
304                 "end_time": all_candidate_windows[wi]["end"],
305                 "shuttle_exchange_count": None,
306                 "shot_count_confidence": "LocalNoAI",
307                 "_candidate_id": candidate_ids[wi],
308             }
309             for wi in handoff_indices
310         ]
311         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
312                     "(no AI handoff).", len(ai_segments))
313     else:
314         handoff_dir = os.path.join(output_base(self.config),
f"chunks_{provider_name}_handoff")
315         os.makedirs(handoff_dir, exist_ok=True)
316         logger.info("Phase 3: %s validation on %d candidate windows...",
provider_name, len(handoff_indices))
317
318     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
319         w_start = max(0, window["start"] - handoff_padding)
320         w_end = min(video_duration, window["end"] + handoff_padding)
321         w_dur = w_end - w_start
322         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
323         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
324             return []
325         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
326         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
327                                             label=f"Handoff {wi+1}")
328         valid = []
329         for seg in segments:
330             try:
331                 seg["start_time"] = float(seg["start_time"]) + w_start
332                 seg["end_time"] = float(seg["end_time"]) + w_start
333                 seg["_candidate_id"] = candidate_ids[wi]
334                 valid.append(seg)
335             except (ValueError, TypeError, KeyError) as e:
336                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
337
338         try:

```

```

339             os.remove(w_path)
340         except OSError:
341             pass
342         return valid
343
344         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
345             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
346                         for wi in handoff_indices]
347             for future in futures:
348                 ai_segments.extend(future.result())
349             try:
350                 os.rmdir(handoff_dir)
351             except OSError:
352                 pass
353             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
354
355             # --- Phase 4: DB population + candidate linking ---
356             segments_data = []
357             for segment in ai_segments:
358                 try:
359                     start = float(segment["start_time"])
360                     end = float(segment["end_time"])
361                     dur = end - start
362                     metadata = {
363                         "shot_count": self._shot_count_from(segment, dur),
364                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
365                         "detector": f"{self.family}-hybrid-{model_name}",
366                     }
367                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
368                     if segment_id:
369                         candidate_id = segment.get("_candidate_id")
370                         if candidate_id is not None:
371                             self.db.link_candidate_to_segment(candidate_id, segment_id)
372                         segments_data.append({
373                             "id": segment_id, "start_time": start, "end_time": end,
374                             "duration": dur, "metadata": metadata,
375                         })
376                 except Exception as e:
377                     logger.warning("Skipping invalid segment struct: %s", e)
378
379             return segments_data
380

```

16. assistant (2026-06-22T09:49:18.510Z)

Perfect! I can now see the critical issue. Let me verify what the actual return structure looks like:

17. user (2026-06-22T09:49:20.147Z)

```

356:         segments_data = []
372:             segments_data.append({
379:         return segments_data

```

18. assistant (2026-06-22T09:49:22.557Z)

Now let me look at the add_play_segment function to see if there's a risk that segment_id can be None:

19. user (2026-06-22T09:49:23.356Z)

```
459:     def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
```

20. user (2026-06-22T09:49:26.079Z)

```
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
463                     cursor.execute(
464                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474             return self._execute_write(
475                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477                 "Failed to add event",
478             )
479
480         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                                   stats: Optional[Dict[str, Any]] = None,
483                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
484             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
485             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
486             try:
487                 with self._connect() as conn:
488                     cursor = conn.cursor()
489                     cursor.execute(
490                         """INSERT INTO candidate_segments
491                         (video_id, detector, chunk_index, start_time, end_time, duration,
492                         confidence, stats, detection_params)
493                         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
494                         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
495                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
496                     )
497                     conn.commit()
498                     return cursor.lastrowid
```

21. assistant (2026-06-22T09:49:37.068Z)

Perfect! Now I have all the evidence I need. Let me trace through the exact issue:

1. ****Line 342 (nightly_reelcount.py)**:** ``db = Database(db_path)`` creates the DB instance
2. ****Line 345**:** ``result = seg_cls(db, config).process_video(video_id, local)`` calls the segmenter
3. ****Line 349**:** ``segs = result.value if hasattr(result, "value") else result`` extracts segs
4. ****Line 351**:** ``return len(segs), intervals, wall, None`` returns the count

Looking at ****trajectory_hybrid.py line 367-375****:

```
`python
segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
if segment_id: # <-- THIS IS THE KEY CHECK
    candidate_id = segment.get("_candidate_id")
    if candidate_id is not None:
        self.db.link_candidate_to_segment(candidate_id, segment_id)
    segments_data.append({...})
...`
```

****THE DEFECT**:** When ``add_play_segment()`` returns ``None`` (due to a DB error, line 471 in database.py), the segment is ****silently dropped**** and NOT added to ``segments_data``. But line 351 of `nightly_reelcount.py` returns ``len(segs)`` which is the count of segments that ****actually made it into segments_data****, not the count of AI-confirmed ``ai_segments`` that were attempted.

This means:

- If a DB insert fails midway through Phase 4, some segments won't make it to ``segments_data``
- The reel count will be ****under-reported**** (missing segments)
- A ****transient DB error will cause a false regression****, since ``len(segs)`` < expected count

The re-run-once guard (line 362 of `nightly_reelcount.py`) can mask a ****persistent**** DB issue if it succeeds on re-run, but only if it's truly transient.

Let me verify this is a real defect in production:

22. user (2026-06-22T09:49:38.317Z)

```
backend/pipeline/segmenters/gemini.py:496:         segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend/pipeline/segmenters/motion.py:201:         segment_id =
self.db.add_play_segment(video_id, start, end, server, receiver, metadata)
backend/pipeline/segmenters/trajectory_hybrid.py:367:         segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend/pipeline/segmenters/yolo_hybrid.py:346:         segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
```

23. user (2026-06-22T09:49:41.119Z)

```
    }

    segment_id = self.db.add_play_segment(video_id, start, end, server, receiver,
metadata)

    if segment_id:
        # Add sub-segment events
        # A: Serve Event (always at start)
        serve_foul = random.random() < 0.15 # 15% chance of serve foul
```

24. assistant (2026-06-22T09:49:43.463Z)

All segmenters use the same pattern. Now let me check if there's any logging or test that would catch this:

25. user (2026-06-22T09:49:44.601Z)

```
./tests/test_nightly_reelcount.py
./tests/test_nightly_regression.py
```


26. user (2026-06-22T09:49:47.166Z)

```
1      """Tests for scripts/nightly_reelcount.py + scripts/nightly_email.py ? the pure cores.
2
3      Fully offline: no GPU, no video, no network, no real corpus. The comparison / cost /
report / email
4      formatting operate on plain dicts + strings, so reel-count regressions, quality drops,
pipeline
5      errors, corpus drift, and the no-creds email skip are all exercised deterministically.
The actual
6      pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on
the VM.
7      """
8      import importlib.util
9      import os
10     import sys
11
12     import pytest
13
14     _SCRIPTS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"scripts")
15
16
17     def _load(modname):
18         spec = importlib.util.spec_from_file_location(modname, os.path.join(_SCRIPTS,
f"{modname}.py"))
19         mod = importlib.util.module_from_spec(spec)
20         sys.modules[modname] = mod # register before exec so dataclass annotations resolve
21         spec.loader.exec_module(mod)
22         return mod
23
24
25     nr = _load("nightly_reelcount")
26     ne = _load("nightly_email")
27
28
29     # ----- #
30     # Builders
31     # ----- #
32     def _video(name, reel_count, ok=True, error=None, quality=None):
33         return {"name": name, "reel_count": reel_count, "ok": ok, "error": error,
34                 "quality": quality, "wall_seconds": 1.0}
35
36
37     def _snap(per_video, segmenter="wasb_hybrid"):
38         return {"segmenter": segmenter, "per_video": per_video,
39                 "cost": {"usd": 0.05, "inr": 4.15, "billed_seconds": 500,
"machine_usd_per_hr": 0.35}}
40
41
42     TOLS = nr.Tolerances()
43
44
45     # ----- #
46     # cost_report (billing wrapper)
47     # ----- #
48     def test_cost_report_math():
49         c = nr.cost_report(billed_seconds=3600, machine_usd_per_hr=0.35,
fx_inr_per_usd=83.0)
50         assert c["usd"] == pytest.approx(0.35, abs=1e-4) # 1hr @ $0.35
51         assert c["inr"] == pytest.approx(0.35 * 83.0, abs=0.01)
52         assert c["billed_seconds"] == 3600
53
54
55     def test_cost_report_zero_for_zero_time():
```

```

56         c = nr.cost_report(billed_seconds=0, machine_usd_per_hr=0.35, fx_inr_per_usd=83.0)
57         assert c["usd"] == 0.0 and c["inr"] == 0.0
58
59
60     # ----- #
61     # summarize_results
62     # ----- #
63     def test_summarize_results():
64         rr = [_video("v1", 5), _video("v2", 3, quality={"tol_f1": 0.4})]
65         snap = nr.summarize_results(rr, "wasb_hybrid", {"usd": 0.1})
66         assert snap["n"] == 2
67         assert snap["per_video"]["v1"]["reel_count"] == 5
68         assert snap["per_video"]["v2"]["quality"]["tol_f1"] == 0.4
69
70
71     # ----- #
72     # compare ? reel count is EXACT
73     # ----- #
74     def test_identical_counts_pass():
75         base = _snap({"v1": {"reel_count": 5, "ok": True}})
76         cur = _snap({"v1": {"reel_count": 5, "ok": True}})
77         res = nr.compare(base, cur, TOLS)
78         assert res.overall_status() == "PASS" and res.exit_code() == 0
79
80
81     def test_reel_count_change_is_regression():
82         base = _snap({"v1": {"reel_count": 5, "ok": True}})
83         cur = _snap({"v1": {"reel_count": 4, "ok": True}})          # one reel vanished
84         res = nr.compare(base, cur, TOLS)
85         assert res.exit_code() == 1
86         r = res.regressions[0]
87         assert r.metric == "reel_count" and r.baseline == 5 and r.current == 4
88
89
90     def test_reel_count_increase_is_also_regression():
91         # ANY change to the count is a regression ? the count must be stable, either
92         direction.
93         base = _snap({"v1": {"reel_count": 5, "ok": True}})
94         cur = _snap({"v1": {"reel_count": 6, "ok": True}})
95         assert nr.compare(base, cur, TOLS).exit_code() == 1
96
97     def test_pipeline_error_is_regression():
98         base = _snap({"v1": {"reel_count": 5, "ok": True}})
99         cur = _snap({"v1": {"reel_count": None, "ok": False, "error": "WASB OOM"}})
100         res = nr.compare(base, cur, TOLS)
101         assert res.exit_code() == 1
102         assert res.regressions[0].metric == "error" and "WASB OOM" in
103         res.regressions[0].detail
104
105     # ----- #
106     # compare ? quality metrics use tolerance
107     # ----- #
108     def test_quality_drop_beyond_tol_is_regression():
109         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
110         cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.30}}})  #
111         ?0.10 ? tol
112         res = nr.compare(base, cur, TOLS)
113         assert res.exit_code() == 1 and res.regressions[0].metric == "tol_f1"
114
115     def test_quality_drop_within_tol_passes():
116         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
117         cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.39}}})  #

```

```

within 0.02
118     assert nr.compare(base, cur, TOLS).exit_code() == 0
119
120
121     def test_quality_rise_is_improvement_not_regression():
122         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.40}}})
123         cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.55}}})
124         res = nr.compare(base, cur, TOLS)
125         assert res.exit_code() == 0
126         assert [f.metric for f in res.improvements] == ["f1@0.5"]
127
128
129     # ----- #
130     # compare ? staleness (segmenter / corpus drift)
131     # ----- #
132     def test_segmenter_change_is_stale():
133         base = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="wasb_hybrid")
134         cur = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="motion")
135         res = nr.compare(base, cur, TOLS)
136         assert res.overall_status() == "STALE" and res.exit_code() == 4
137
138
139     def test_corpus_drift_is_stale_but_checks_intersection():
140         base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok":
True}}})
141         cur = _snap({"v1": {"reel_count": 9, "ok": True}, "v3": {"reel_count": 1, "ok":
True}}})
142         res = nr.compare(base, cur, TOLS)
143         assert "v3" in res.added and "v2" in res.removed
144         # v1 regressed on the intersection ? regression wins over stale (exit 1)
145         assert res.exit_code() == 1
146         assert any(f.video == "v1" and f.metric == "reel_count" for f in res.regressions)
147
148
149     def test_corpus_drift_clean_intersection_is_stale_exit_4():
150         base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok":
True}}})
151         cur = _snap({"v1": {"reel_count": 5, "ok": True}})          # v2 dropped, v1 unchanged
152         res = nr.compare(base, cur, TOLS)
153         assert res.exit_code() == 4 and res.overall_status() == "STALE"
154
155
156     # ----- #
157     # report formatting (smoke)
158     # ----- #
159     def test_format_report_has_status_cost_and_regression():
160         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.4,
"f1@0.5": 0.5}}})
161         base["git_commit"] = "aaa"
162         base["generated_at"] = "2026-06-21"
163         cur = _snap({"v1": {"reel_count": 3, "ok": True, "quality": {"tol_f1": 0.4,
"f1@0.5": 0.5}}})
164         res = nr.compare(base, cur, TOLS)
165         rpt = nr.format_report(base, cur, res, "2026-06-22", "bbb")
166         assert "Nightly engine regression" in rpt
167         assert "REGRESSION" in rpt
168         assert "Run cost:" in rpt and "?" in rpt
169         assert "reel_count" in rpt
170         assert "v1" in rpt
171
172
173     def test_format_report_pass_badge():
174         base = _snap({"v1": {"reel_count": 5, "ok": True}})
175         cur = _snap({"v1": {"reel_count": 5, "ok": True}})
176         res = nr.compare(base, cur, TOLS)

```

```

177         assert "PASS" in nr.format_report(base, cur, res, "2026-06-22", "bbb")
178
179
180     # ----- #
181     # email ? build + creds resolution + graceful skip
182     # ----- #
183     def test_email_build_message():
184         msg = ne.build_message("subj", "body **md**", "from@x.com", "to@y.com")
185         assert msg["Subject"] == "subj" and msg["To"] == "to@y.com" and msg["From"] ==
"from@x.com"
186         assert "body **md**" in msg.get_content()
187
188
189     def test_email_resolve_secret_from_env():
190         assert ne.resolve_secret({"NIGHTLY_SMTP_PASS": "hunter2"}) == "hunter2"
191
192
193     def test_email_resolve_secret_none_when_absent():
194         assert ne.resolve_secret({}) is None
195
196
197     def test_email_send_skips_without_creds():
198         # No password resolvable ? graceful skip (False), never raises, no network.
199         assert ne.send_report("s", "b", to="x@y.com", env={"NIGHTLY_EMAIL_TO": "x@y.com"})
is False
200

```

27. assistant (2026-06-22T09:49:49.551Z)

The tests don't cover the case where `add\_play\_segment` returns None. Let me check the nightly_reelcount.py for actual integration test coverage:

28. user (2026-06-22T09:49:52.002Z)

6:pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on the VM.

29. assistant (2026-06-22T09:49:54.384Z)

Confirmed: the tests **do not** cover the actual `run\_one\_video` integration. Let me check if there are any integration tests:

30. user (2026-06-22T09:49:55.279Z)

tests/test_nightly_reelcount.py:6:pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on the VM.

31. assistant (2026-06-22T09:49:57.575Z)

No actual integration tests. Now let me check the documentation comment around the transient flake to understand the context:

32. user (2026-06-22T09:49:58.466Z)

```

1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3     Runs the configured + checkpointed pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the number of reels does
not change
5     vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6     trajectories, the windowing/scorer only), this one runs the full pipeline on the real
model, so it

```

```

7         needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8         owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10        What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11        ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12        Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13        **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14        ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15        ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16        ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18        Extensible pattern (add a video = one manifest row + re-freeze):
19        ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20        ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)

```

33. assistant (2026-06-22T09:50:06.718Z)

Perfect! Line 13 explicitly mentions "a transient DB-insert-None" as a ****known flake**** that the re-run-once guard is designed to handle. Let me verify the guard is actually implemented:

Looking back at lines 362-366 of `nightly_reelcount.py`:

```

``python
if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
    count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
    ...
    if err2 is None and count2 is not None:
        count, intervals = count2, intervals2
...

```

The guard is implemented to catch ****transient**** failures (segments missing due to DB insertion flakes). However, there's a critical risk:

****The defect:** The reel count is derived from `len(segments_data)` which only includes successfully-inserted segments. ****** If `add_play_segment()` silently fails for ****some**** segments but not all, those segments are lost from the count.

The re-run-once guard can only catch this if:

1. The flake is ****truly transient**** (won't happen on the second run)
2. The baseline is set (baseline_count is not None)

But if the DB error is ****semi-persistent**** (e.g., a database lock that clears on retry) or if the ****first baseline run**** also had silent segment losses, the baseline itself is corrupted.

Let me verify if the baseline generation process has the same risk:

34. user (2026-06-22T09:50:07.989Z)

```

440        "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441        "configured+checkpointed engine + the manifest corpus.")
442        out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443        os.makedirs(os.path.dirname(path), exist_ok=True)
444        tmp = path + ".tmp"
445        with open(tmp, "w", encoding="utf-8") as fh:

```

```

446         json.dump(out, fh, indent=2, sort_keys=True)
447         fh.write("\n")
448     os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
453 quality + cost.")
454         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
455         p.add_argument("--baseline", default=DEFAULT_BASELINE)
456         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
457         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
458         p.add_argument("--vm-uptime-seconds", type=float, default=None,
459             help="authoritative VM uptime (boot?delete) for the cost line; "
460             "defaults to the in-process pipeline wall time (a lower bound)")
461         p.add_argument("--no-rerun-guard", action="store_true",
462             help="disable the re-run-once-on-mismatch guard (the DB-drop flake
463 mitigation)")
464         p.add_argument("--update-baseline", action="store_true",
465             help="freeze the current reel counts + quality as the new baseline
466 instead of checking")
467         p.add_argument("--email", action="store_true", help="email the report (see
468 scripts/nightly_email.py)")
469         p.add_argument("--email-to", default=None, help="override the report recipient")
470         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
471 for tests)")
472         p.add_argument("--no-report", action="store_true", help="skip writing the dated
473 report file")
474         return p
475
476
477     def main(argv: Optional[List[str]] = None) -> int:
478         import logging
479         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
480         log = logging.getLogger("nightly_reelcount")
481         args = build_parser().parse_args(argv)
482         tols = Tolerances(quality_tol=args.quality_tol)
483         date = args.date or _dt.date.today().isoformat()
484
485         if not os.path.exists(args.manifest):
486             log.error("manifest not found: %s", args.manifest)
487             return 2
488         manifest = load_manifest(args.manifest)
489         baseline = load_baseline(args.baseline)
490
491         try:
492             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
493 args.no_rerun_guard)
494         except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
495 never a false "pass"
496             log.error("run failed: %s", e)
497             return 2
498         if not run_results:
499             log.error("no videos in manifest %s", args.manifest)
500             return 2
501
502         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
503 total_wall
504         cost = cost_report(
505             billed_seconds=billed,
506             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
507 DEFAULT_MACHINE_USD_PER_HR)),
508             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
509             gpu_seconds=total_wall,
510         )

```

```

501     segmenter = manifest.get("segmenter", "motion")
502     snapshot = summarize_results(run_results, segmenter, cost)
503     snapshot["git_commit"] = _git_head()
504     snapshot["generated_at"] = date
505
506     if args.update_baseline:
507         save_baseline(args.baseline, snapshot)
508         log.info("baseline frozen ? %s", args.baseline)
509         for r in run_results:
510             log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511         return 0
512
513     if baseline is None:
514         log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515         return 3
516
517     result = compare(baseline, snapshot, tols)
518     report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520     if not args.no_report:
521         os.makedirs(args.report_dir, exist_ok=True)
522         md_path = os.path.join(args.report_dir, f"{date}.md")
523         with open(md_path, "w", encoding="utf-8") as fh:
524             fh.write(report + "\n")
525         with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526             json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                       "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                       fh, indent=2, sort_keys=True)
529         log.info("report ? %s", md_path)
530
531     print(report)
532
533     if args.email:
534         try:
535             from scripts.nightly_email import send_report
536             subject = f"[KheISutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537             sent = send_report(subject, report, to=args.email_to)
538             log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539         except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict

```

35. assistant (2026-06-22T09:50:18.800Z)

Now let me look at the core issue more directly. The problem is:

```

**In trajectory_hybrid.py line 367, when `add_play_segment()` returns None due to a DB error:**
- The segment is NOT added to `segments_data`
- But `len(segs)` in nightly_reelcount.py:351 will be **less than** the number of AI-confirmed
segments
- The reel count is under-reported

```

Let me create a defect case and verify this is production-critical:

```

**Scenario 1 (the documented transient flake):**
- Run 1: DB lock on segment 3 ? add_play_segment returns None ? segments_data has 9 segs (not
10) ? baseline frozen at 9
- Baseline is now corrupted but passes

```

****Scenario 2 (the actual regression risk):****

- Run 1 (baseline freeze): 10 segments, all inserted successfully ? baseline = 10 ?
- Run 2 (nightly check, after a code change): 10 AI segments found, but DB error on segment 7 ? segments_data has 9 ? count = 9
- Regression detected (10 ? 9) even though the engine is working perfectly

The re-run guard would catch scenario 2 IF segment 7 inserts successfully on run 2. But if the error is semi-persistent (e.g., a race condition in DB connection pooling), both runs fail identically and the error is reported but the diagnosis is wrong.

****The ROOT CAUSE:**** The code conflates "AI-detected segments" (correct count) with "segments successfully persisted to DB" (which can be less due to errors). The reel count should be based on the former, not the latter.

Now I'll compile my findings:

36. user (2026-06-22T09:50:29.883Z)

Structured output provided successfully